

Ph. D. Juan David Martínez Vargas
Ph. D. Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

Lecture 09

Deep Learning

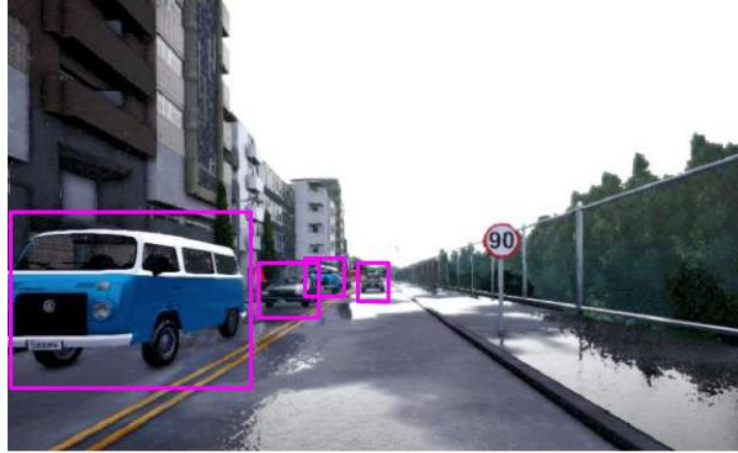
Agenda

- **Unet**
- **Speckle**
- **Noise MetreCs**
- **Ejemplo**

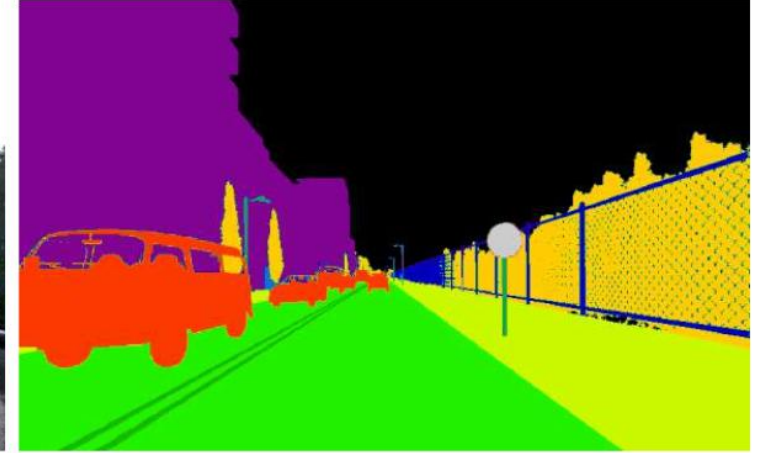
Object detection vs Semantic Segmentation



Input image



Object Detection

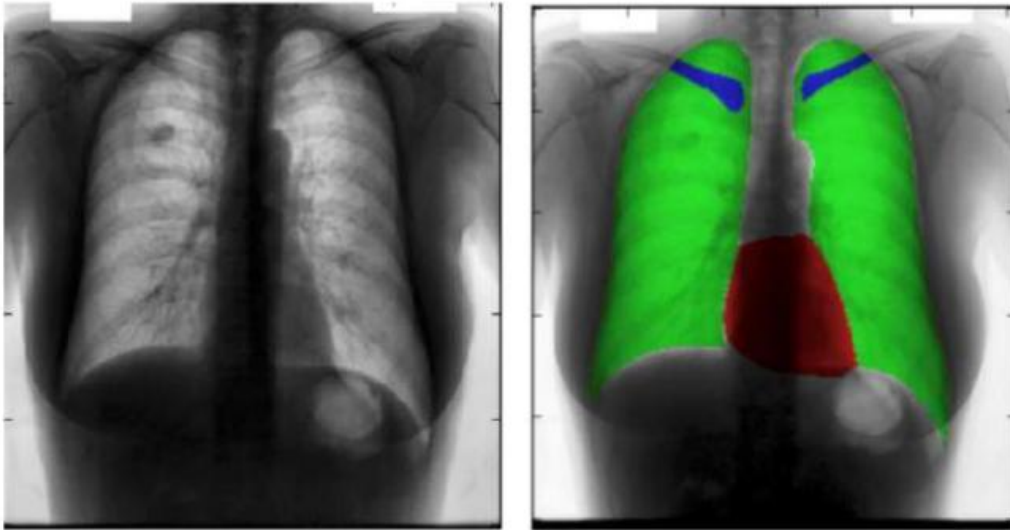


Semantic Segmentation

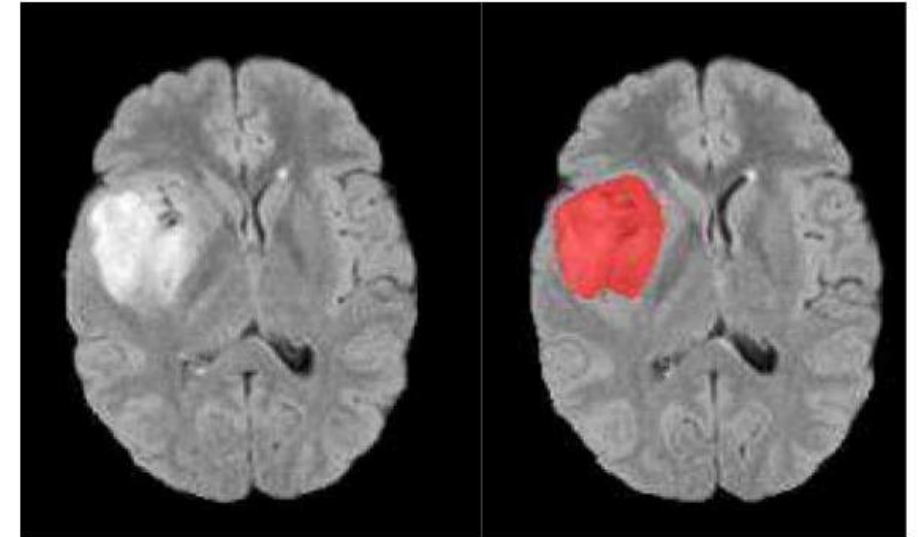
En una arquitectura encoder-decoder, el encoder reduce progresivamente la resolución espacial de la imagen mientras aumenta la riqueza de sus representaciones.

Este proceso permite aprender características cada vez más abstractas, pero también puede hacer que se pierda información espacial fina: **bordes, posiciones y detalles locales**.

Object detection vs Semantic Segmentation



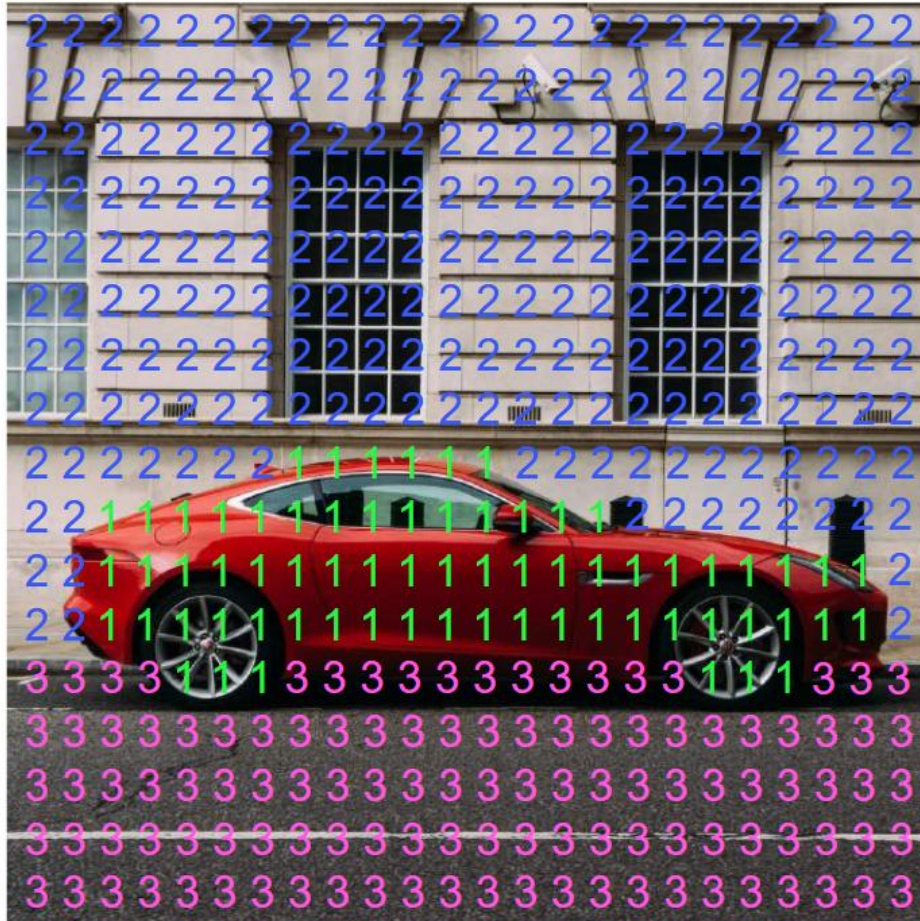
Chest X-Ray



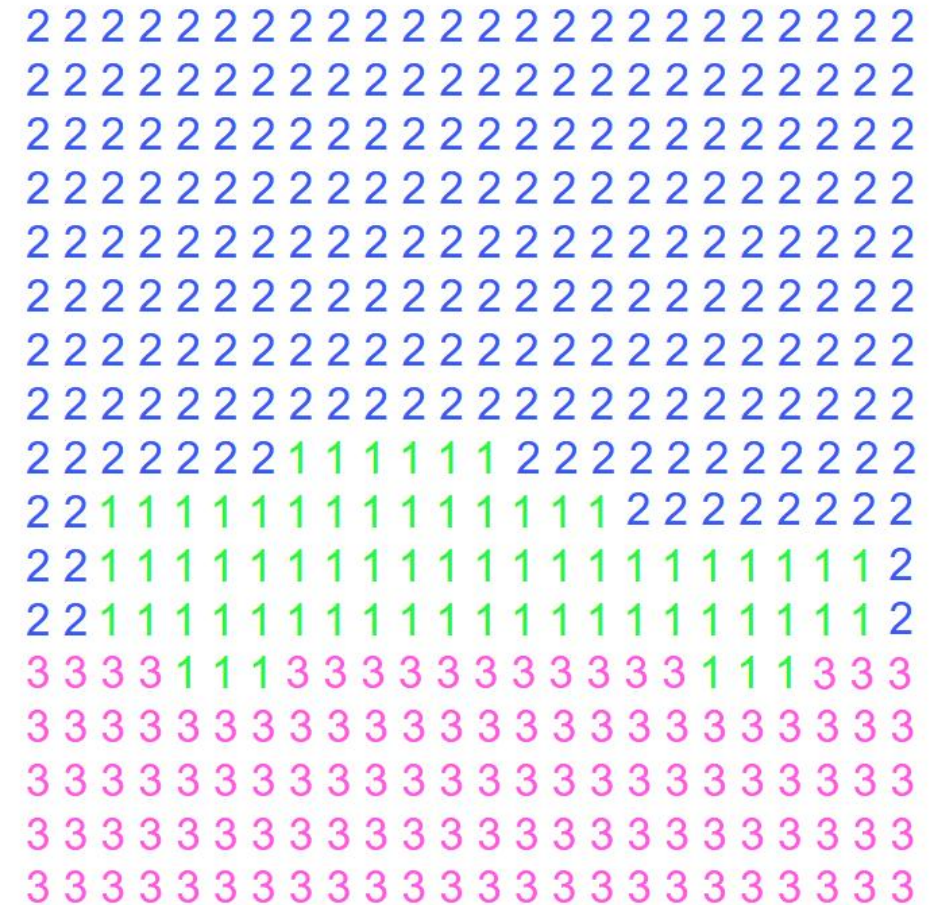
Brain MRI

[Novikov et al., 2017, Fully Convolutional Architectures for Multi-Class Segmentation in Chest Radiographs]

[Dong et al., 2017, Automatic Brain Tumor Detection and Segmentation Using U-Net Based Fully Convolutional Networks]

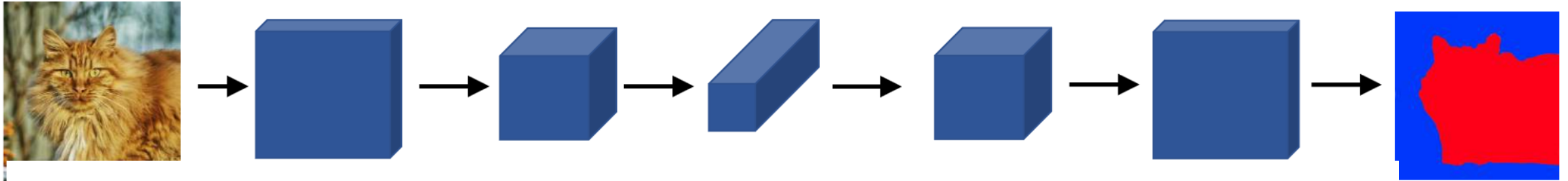


1. Car
2. Building
3. Road

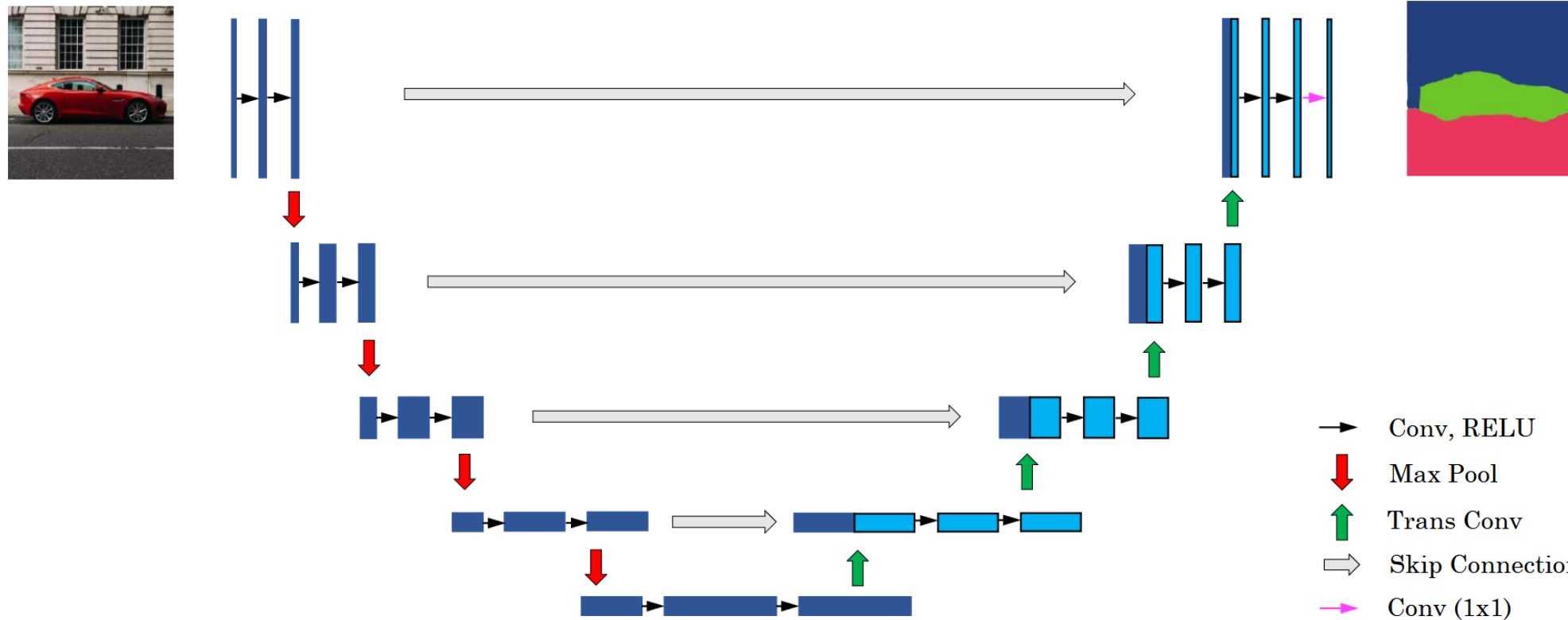


Segmentation Map

U-Net



U-Net



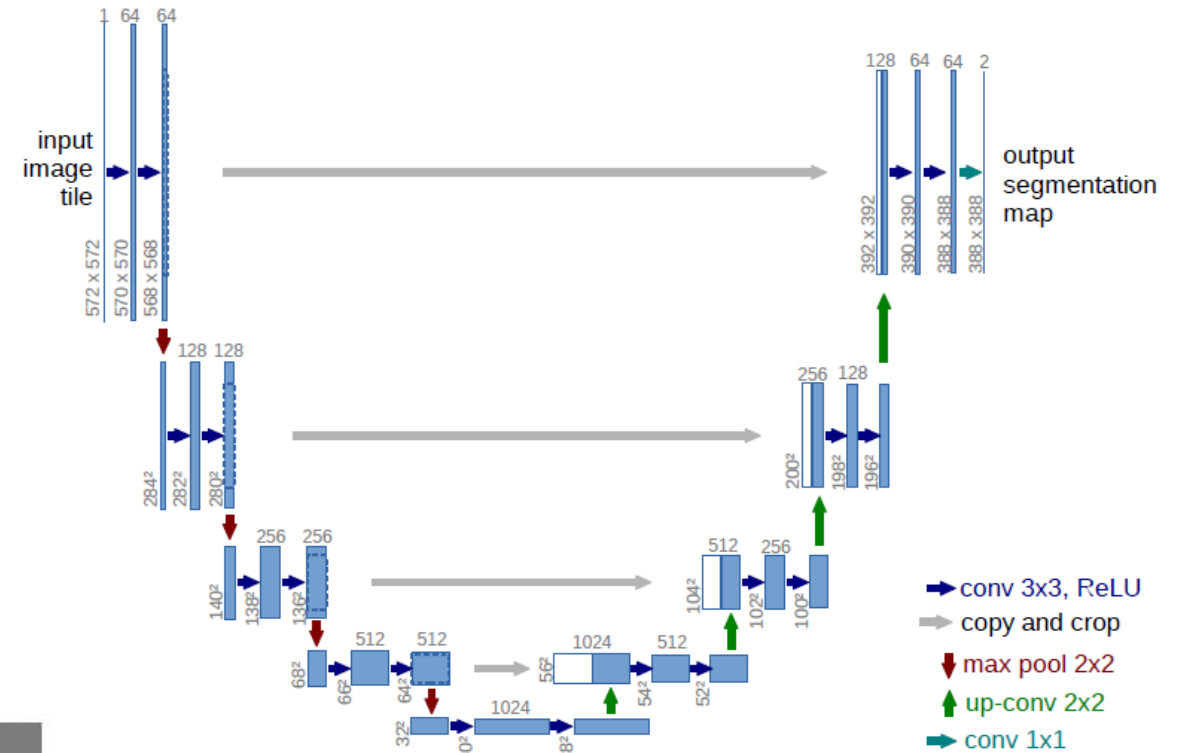
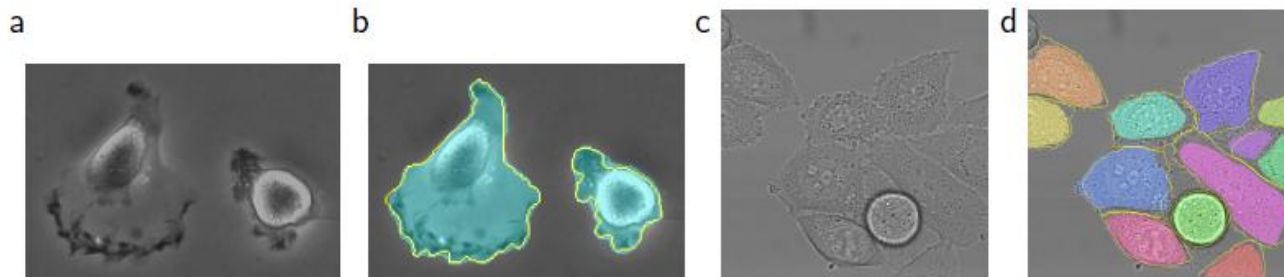
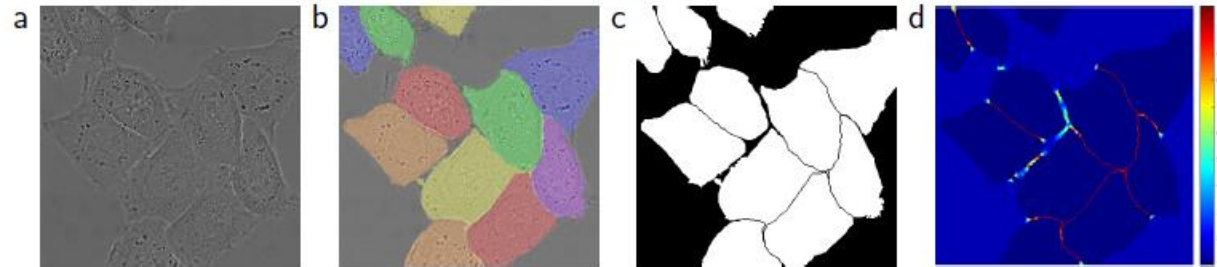
U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

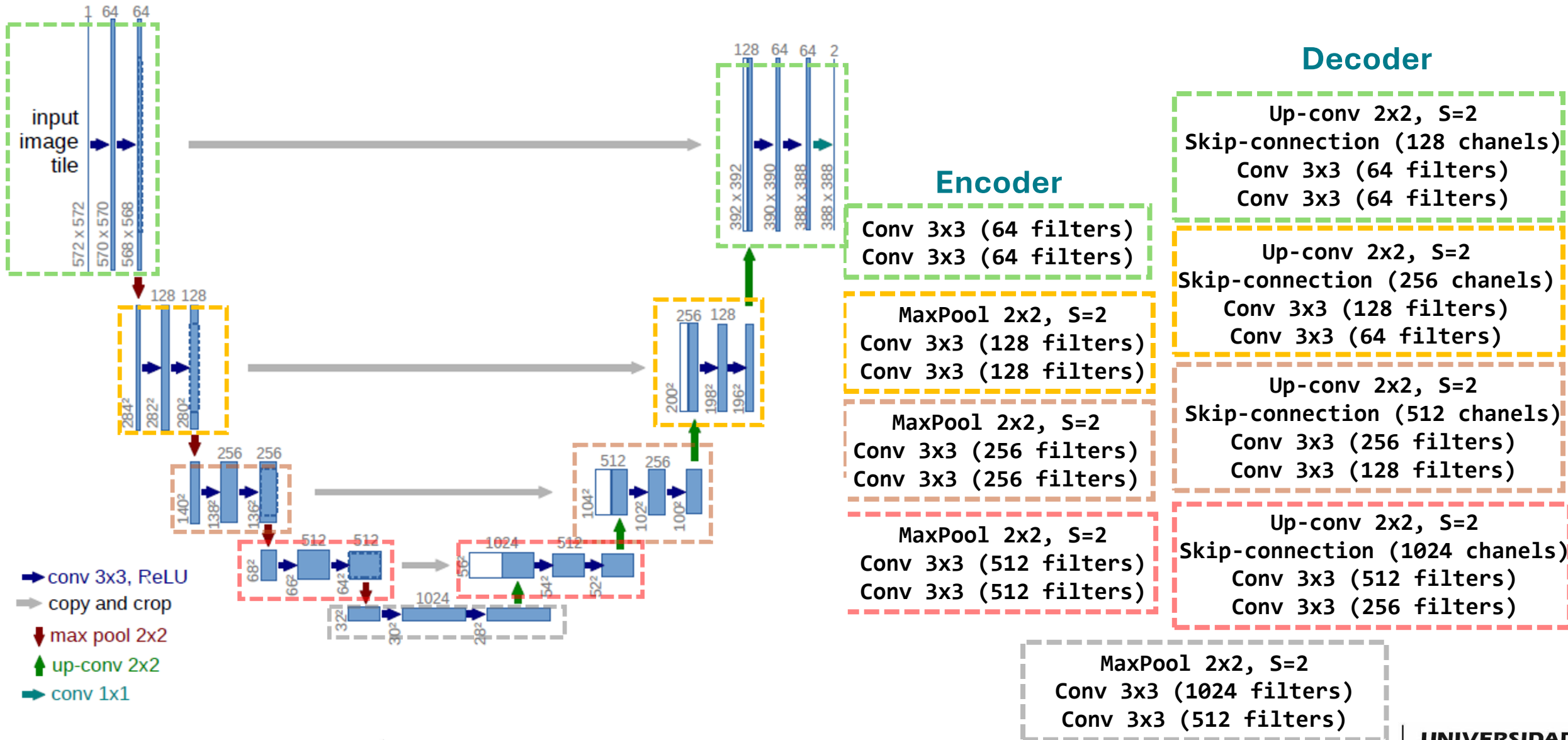
Computer Science Department and BIOS Centre for Biological Signalling Studies,
University of Freiburg, Germany

ronneber@informatik.uni-freiburg.de,

WWW home page: <http://lmb.informatik.uni-freiburg.de/>



U-Net Architecture



U-Net Architecture

Skip connection → transporta información

Concatenation → reúne información

Convolution → aprende a fusionar información

La skip connection transporta los feature maps del encoder y después estos se concatenan con los feature maps del decoder

Encoder

Dimensión espacial



Para un nivel l :

$$E_l \in \mathbb{R}^{H_l \times W_l \times C_l}$$



Representación
almacenada en el encoder

Número de
canales

Decoder

Dimensión espacial



$$D_{l+1} \in \mathbb{R}^{\frac{H_l}{2} \times \frac{W_l}{2} \times 2C_l}$$



Representación
almacenada en el decoder

Número de
canales

up-convolution

$$U_l = \mathcal{U}_l(D_{l+1})$$



Operador encargado de
aumentar la resolución
espacial

$$U_l \in \mathbb{R}^{H_l \times W_l \times C_l}$$

U-Net Architecture

El primer requisito para usar la skip connection es la compatibilidad espacial: encoder y decoder deben referirse a la misma escala $H \times W$.

$$E_l \in \mathbb{R}^{H_l \times W_l \times C_l} \quad \boxed{U_l \in \mathbb{R}^{H_l \times W_l \times C_l}}$$

Concatenar

$$F_l = \text{Concat}(E_l, U_l)$$

$$F_l \in \mathbb{R}^{H_l \times W_l \times 2C_l}$$

Convolucion **Aprende cómo fusionar esa información**

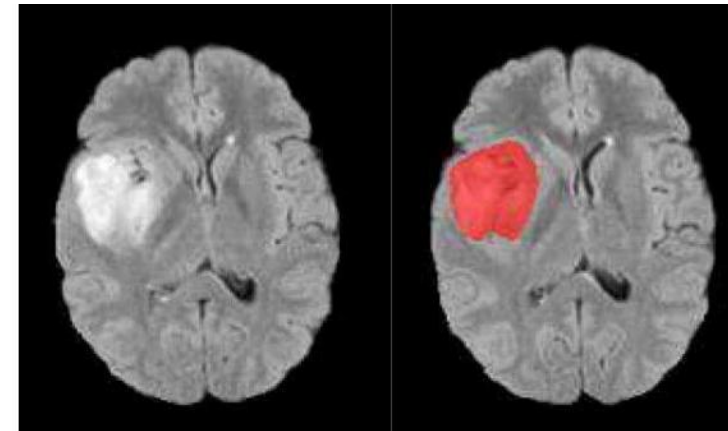
$$D_l = \sigma(\text{Conv}(F_l; W_l) + b_l)$$

$$D_l \in \mathbb{R}^{H_l \times W_l \times C_l}$$

Vector A: [1, 2, 3]

$\text{Concat}(A, B) = [1, 2, 3, 4, 5]$

Vector B: [4, 5]



U-Net Architecture

Skip connection → transporta información

Concatenation → reúne información

Convolution → aprende a fusionar información

La skip connection transporta los feature maps del encoder y después estos se concatenan con los feature maps del decoder

Encoder

Dimensión espacial



Para un nivel l :

$$E_l \in \mathbb{R}^{H_l \times W_l \times C_l}$$



Representación
almacenada en el encoder

Número de
canales

Decoder

Dimensión espacial



$$D_{l+1} \in \mathbb{R}^{\frac{H_l}{2} \times \frac{W_l}{2} \times 2C_l}$$



Representación
almacenada en el decoder

Número de
canales

up-convolution

$$U_l = \mathcal{U}_l(D_{l+1})$$



Operador encargado de
aumentar la resolución
espacial

$$U_l \in \mathbb{R}^{H_l \times W_l \times C_l}$$

U-Net - Example

```
class ConvBlock(nn.Module):  
    """Convolutional block: Conv → BatchNorm → ReLU (×2)"""
```

← New class (layer-like behavior)

```
def __init__(self, in_channels, out_channels):  
    super().__init__()
```

← Arguments

```
    self.block = nn.Sequential(  
        nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),  
        nn.BatchNorm2d(out_channels),  
        nn.ReLU(inplace=True),  
        nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),  
        nn.BatchNorm2d(out_channels),  
        nn.ReLU(inplace=True),  
    )
```

← Group several layers in sequence using nn.Sequential.

← Convolution applies learnable filters to the image or feature map to detect local patterns

```
def forward(self, x):  
    return self.block(x)
```

← This method defines how information flows forward.

U-Net - Example

```
class SpeckleDenoiseUnet(nn.Module):
```

```
    """
```

```
def __init__(self, in_channels=1, features=[32, 64, 128, 256]):  
    super().__init__()
```

```
    # — Encoder —
```

```
self.enc1 = ConvBlock(in_channels, features[0])    # 128 → 128  
self.enc2 = ConvBlock(features[0], features[1])   # 64  → 64  
self.enc3 = ConvBlock(features[1], features[2])   # 32  → 32  
self.enc4 = ConvBlock(features[2], features[3])   # 16  → 16
```

```
self.pool = nn.MaxPool2d(2, 2)
```

```
    # — Bottleneck —
```

```
self.bottleneck = ConvBlock(features[3], features[3] * 2) # 8×8
```

Feature maps

Encoder

Max Pooling

It's the most compact representation of the image. The network attempts to preserve the important information from the original image

U-Net - Example

```
# — Decoder —
self.up4 = nn.ConvTranspose2d(features[3]*2, features[3], kernel_size=2, stride=2)
self.dec4 = ConvBlock(features[3]*2, features[3]) # +skip

self.up3 = nn.ConvTranspose2d(features[3], features[2], kernel_size=2, stride=2)
self.dec3 = ConvBlock(features[2]*2, features[2]) # +skip

self.up2 = nn.ConvTranspose2d(features[2], features[1], kernel_size=2, stride=2)
self.dec2 = ConvBlock(features[1]*2, features[1]) # +skip

self.up1 = nn.ConvTranspose2d(features[1], features[0], kernel_size=2, stride=2)
self.dec1 = ConvBlock(features[0]*2, features[0]) # +skip
```

Decoder

Increase the spatial
resolution of the image

This is what doubles the
resolution.

$$H_{out} = (H_{in} - 1) \times stride + kernel - 2 \times padding$$

The ConvTranspose2d layer performs learned upsampling, increasing the spatial resolution of the feature maps from 8×8 to 16×16. The result is then concatenated with the corresponding encoder feature maps (skip connection), doubling the number of channels. A convolutional block then processes this combined information to refine the reconstruction.

U-Net - Example

```
# — Decoder —  
self.up4 = nn.ConvTranspose2d(features[3]*2, features[3], kernel_size=2, stride=2)  
self.dec4 = ConvBlock(features[3]*2, features[3]) # +skip  
  
self.up3 = nn.ConvTranspose2d(features[3], features[2], kernel_size=2, stride=2)  
self.dec3 = ConvBlock(features[2]*2, features[2]) # +skip  
  
self.up2 = nn.ConvTranspose2d(features[2], features[1], kernel_size=2, stride=2)  
self.dec2 = ConvBlock(features[1]*2, features[1]) # +skip  
  
self.up1 = nn.ConvTranspose2d(features[1], features[0], kernel_size=2, stride=2)  
self.dec1 = ConvBlock(features[0]*2, features[0]) # +skip
```

Decoder

Increase the spatial
resolution of the image

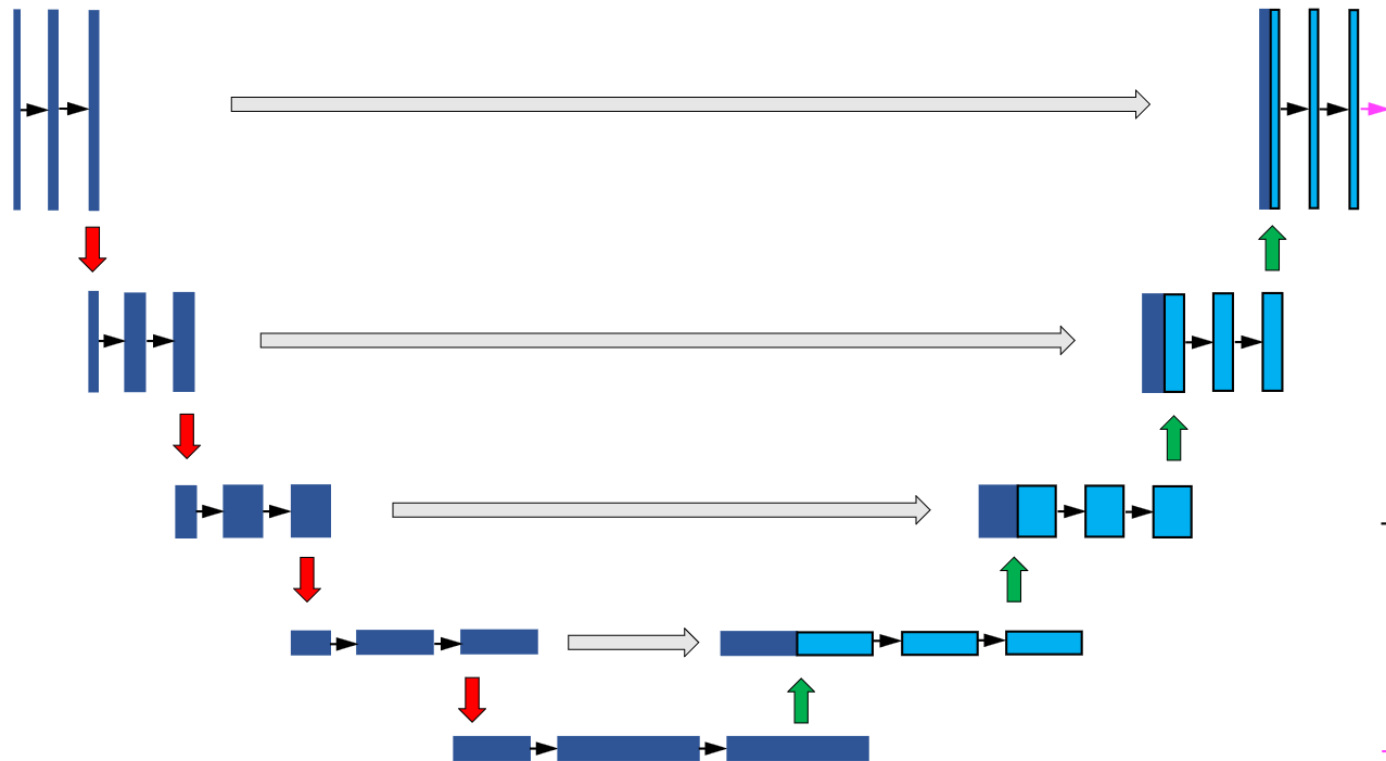
This is what doubles the
resolution.

$$H_{out} = (H_{in} - 1) \times stride + kernel - 2 \times padding$$

The ConvTranspose2d layer performs learned upsampling, increasing the spatial resolution of the feature maps from 8×8 to 16×16. The result is then concatenated with the corresponding encoder feature maps (skip connection), doubling the number of channels. A convolutional block then processes this combined information to refine the reconstruction.

U-Net - Example

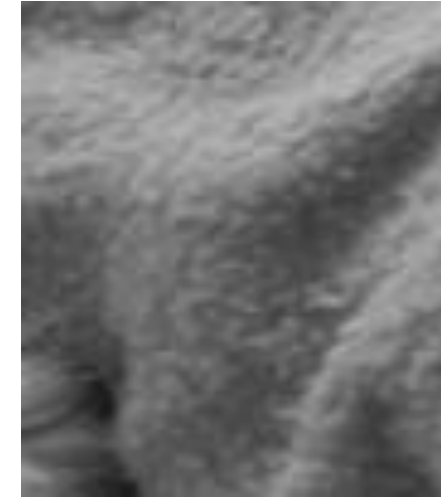
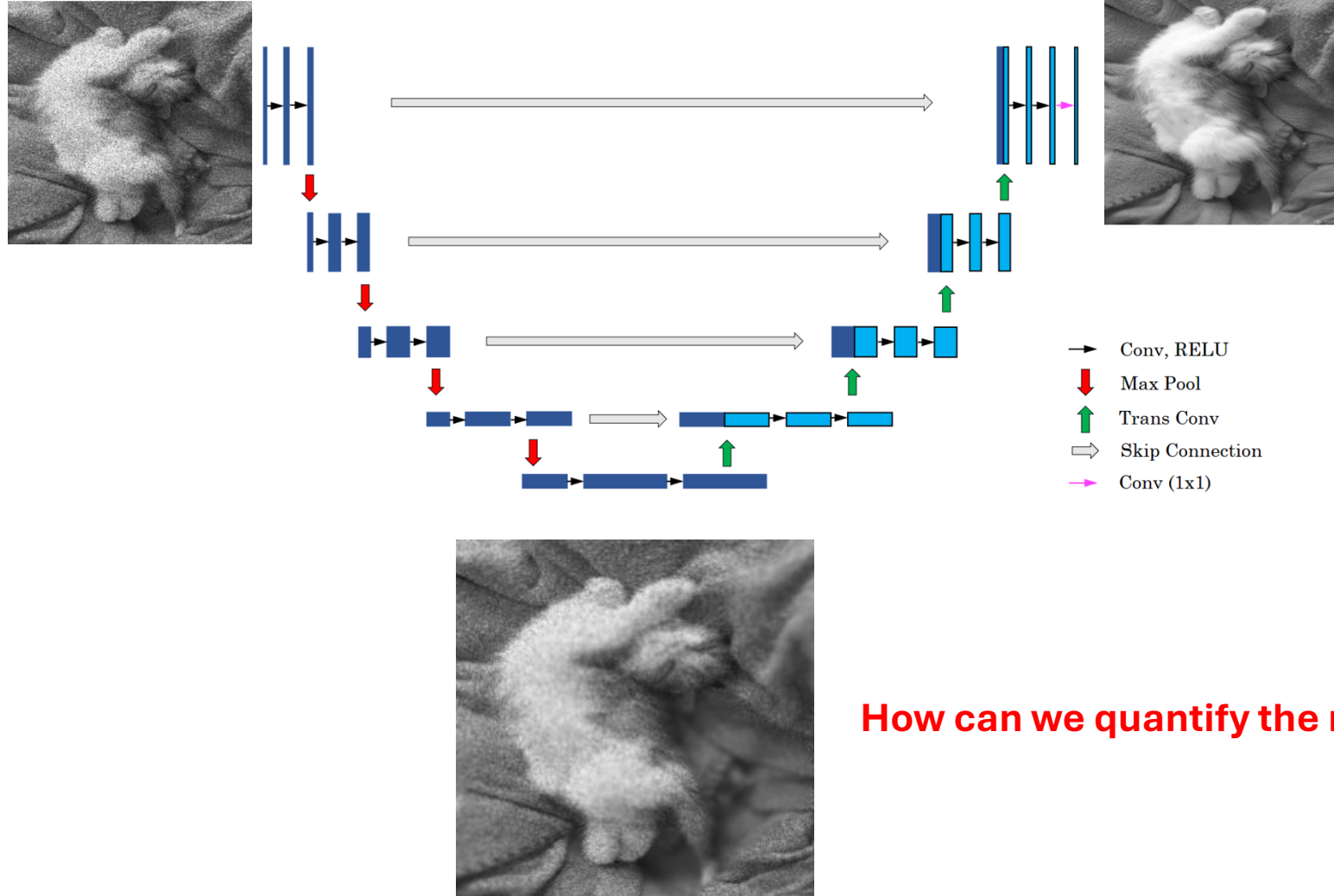
U-Net



- Conv, RELU
- ↓ Max Pool
- ↑ Trans Conv
- Skip Connection
- Conv (1x1)

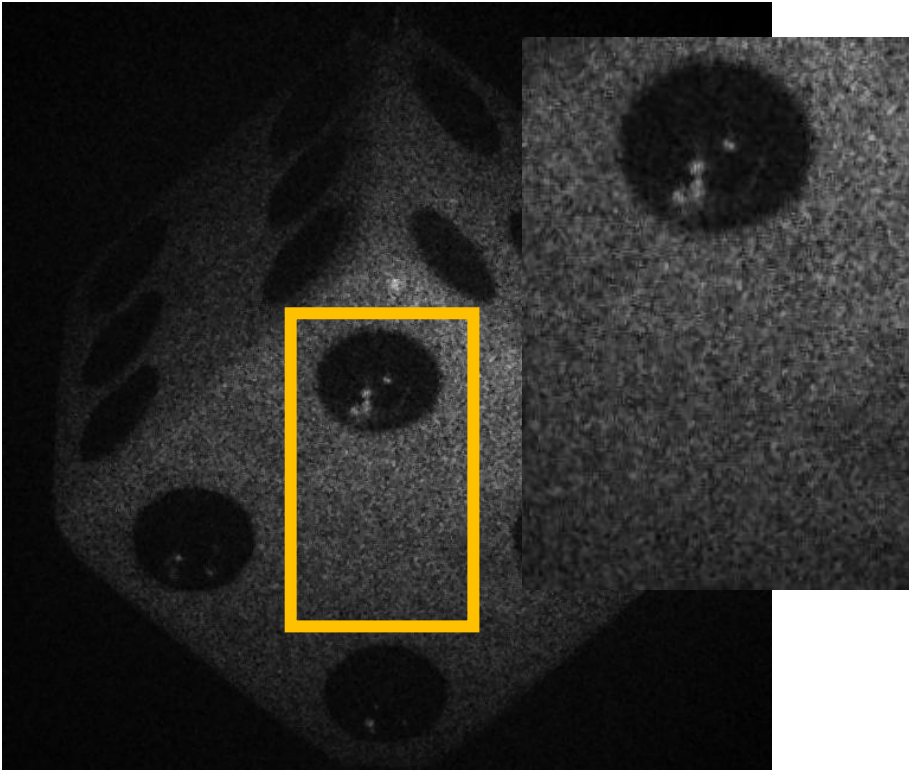
U-Net - Example

U-Net

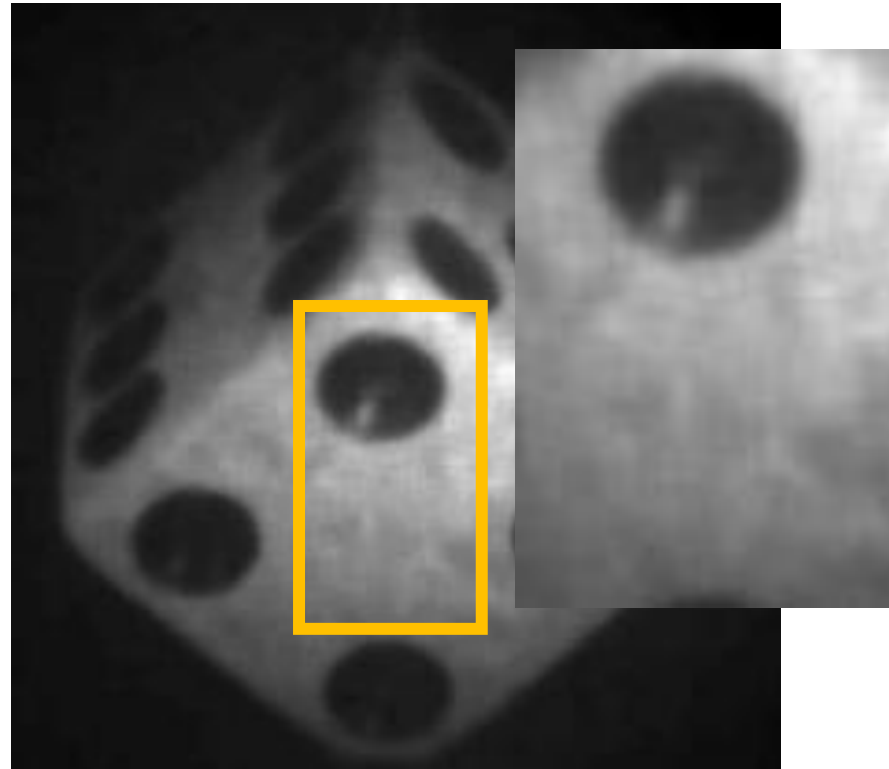


Ruido en imágenes

Imagen con ruido

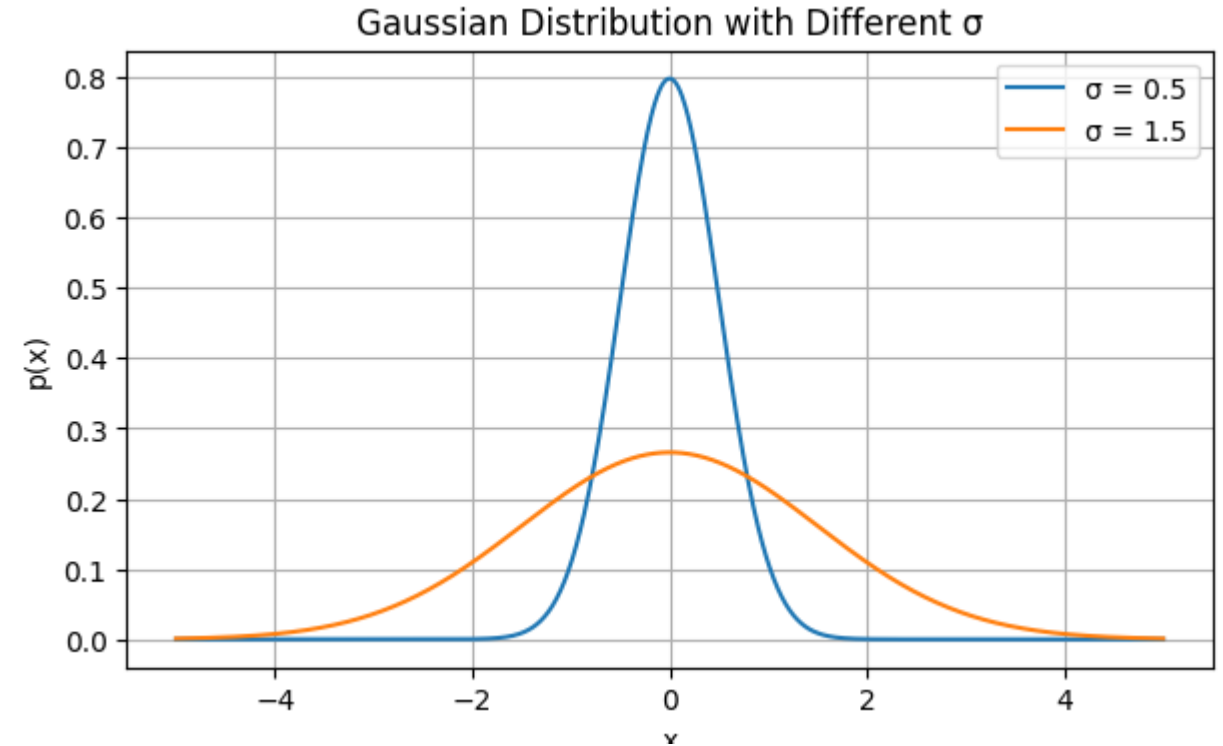
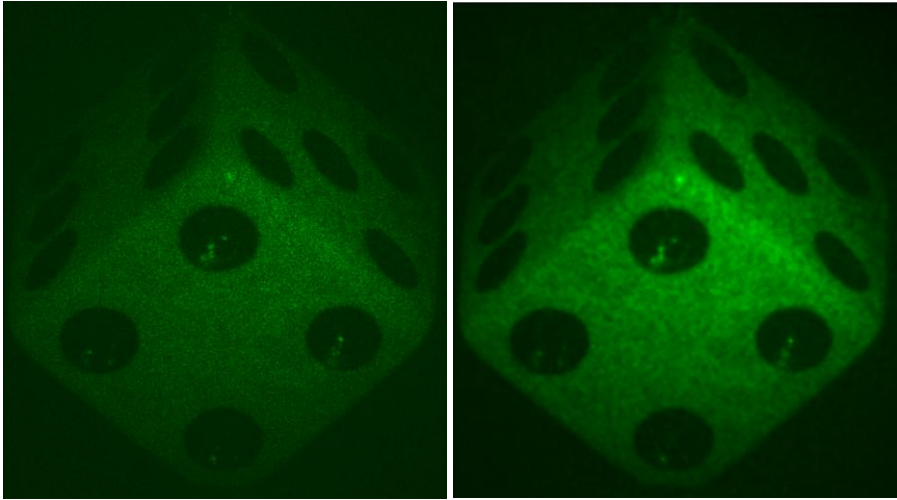


Median-filtered Imagen sin ruido



The application of the median filter reduces the noise as well as the resultant image is more blurred than the input imagen (i.e., reduction of the resolution limit).

Ruido en imágenes



$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{\sigma^2}\right)$$

How can we quantify the noise?

Ruido en imágenes

How can we quantify the noise?



Peak Signal-to-Noise Ratio (PSNR)

Peak Signal-to-Noise Ratio (PSNR) is a measurement used to evaluate the quality of reconstructed or enhanced images by comparing the maximum possible signal strength to the noise that affects its fidelity. It is expressed in decibels (dB) because it is a logarithmic relationship between signal and error.

$$PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

$$MSE = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N (I_D(x, y) - I_S(x, y))^2$$

Diagram illustrating the Mean Squared Error (MSE) formula with annotations:

- MN : Image dimensions
- $I_D(x, y)$: Pixel intensity in the clean image
- $I_S(x, y)$: Pixel intensity in the noisy image

| PSNR |

| < 20 dB |
| 20–30 dB |
| 30–40 dB |
| > 40 dB |

Interpretation

severely degraded image
visible degradation
good quality
almost indistinguishable from the original

Ruido en imágenes

How can we quantify the noise?



Speckle Index (SI)

A speckle index, often referred to as a Speckle Contrast measures the intensity of granular noise (speckle) in imaging systems such as Synthetic Aperture Radar (SAR) or ultrasound.

$$SI = \frac{\sigma}{\mu}$$

← Standard deviation intensity

← Average intensity

$$\sigma = \sqrt{\frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N (I(x, y) - \mu)^2}$$

$$\mu = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N I(x, y)$$

Ruido en imágenes

How can we quantify the noise?



Equivalent Number of Looks (ENL)

The Equivalent Number of Looks (ENL) is a critical parameter in Synthetic Aperture Radar (SAR) imaging that measures the level of speckle noise reduction, with higher values indicating lower noise and better quality. It represents the actual degree of averaging performed during imaging, often calculated in homogeneous, non-textured regions

$$ENL = \left(\frac{\sigma}{\mu} \right)^2$$

← Standard deviation intensity

← Average intensity

$$\sigma = \sqrt{\frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N (I(x, y) - \mu)^2}$$

$$\mu = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N I(x, y)$$

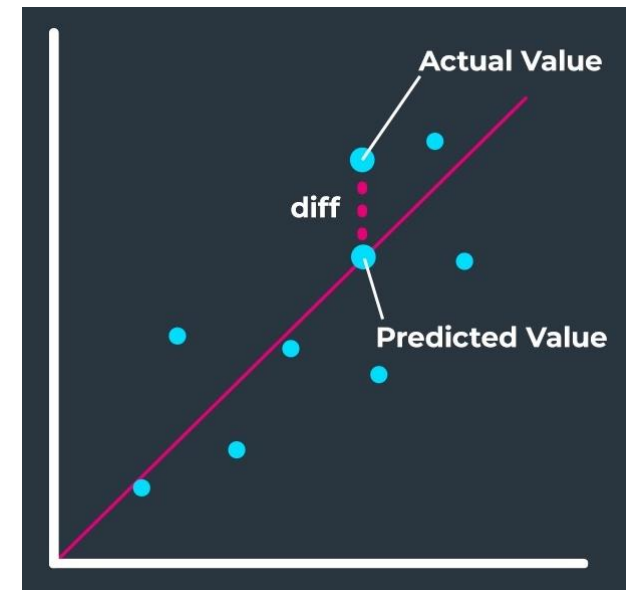
U-Net - Example

$$\mathcal{L}_t = \alpha \mathcal{L}_{\text{MSE}} + \beta \mathcal{L}_{\text{SSIM}}$$



MSE treats an image as if it were just a collection of numbers, pixel by pixel. It doesn't properly understand that an image has: edges, shapes, textures, local patterns, spatial relationships between neighbors.

$$MSE = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N (I_D(x, y) - I_S(x, y))^2$$



U-Net - Example

$$\mathcal{L}_t = \alpha \mathcal{L}_{\text{MSE}} + \beta \mathcal{L}_{\text{SSIM}}$$



SSIM Structural Similarity Index Measure. It's a metric designed to measure the similarity between two images in a way that more closely approximates human visual perception. Instead of just looking at pixel-by-pixel differences, it compares: luminance, contrast, structure.

$$SSIM(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

$$0 \leq SSIM(x, \hat{x}) \leq 1$$

Promedio

$$\mu_x = \frac{1}{N} \sum_{i=1}^N x_i \quad \mu_{\hat{x}} = \frac{1}{N} \sum_{i=1}^N \hat{x}_i$$

Varianza y desviación estándar

$$\sigma_x^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)^2 \quad \sigma_{\hat{x}}^2 = \frac{1}{N} \sum_{i=1}^N (\hat{x}_i - \mu_{\hat{x}})^2$$

Covarianza

$$\sigma_{x\hat{x}} = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)(\hat{x}_i - \mu_{\hat{x}})$$

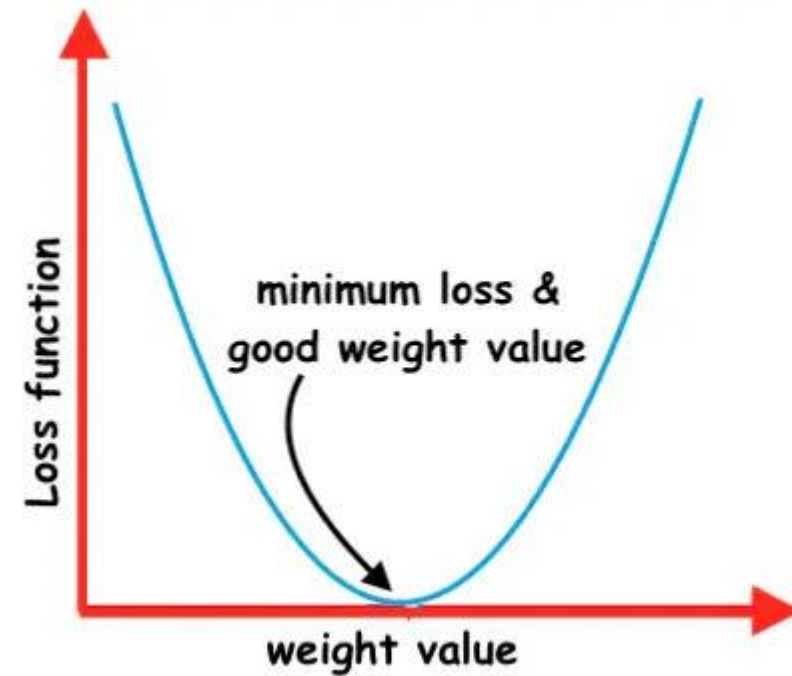
U-Net - Example

$$\mathcal{L}_t = \alpha \mathcal{L}_{\text{MSE}} + \beta \mathcal{L}_{\text{SSIM}}$$

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

$$\text{MSE} = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N (I_D(x, y) - I_S(x, y))^2$$

$$\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}$$



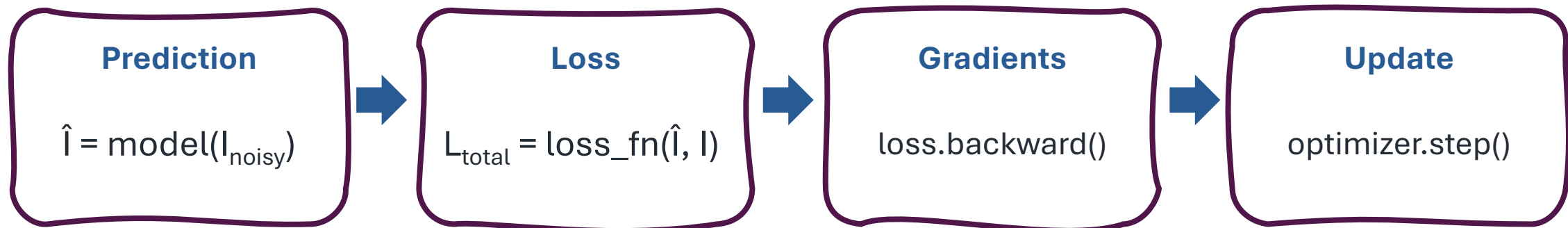
U-Net - Example

$$\mathcal{L}_t = \alpha \mathcal{L}_{\text{MSE}} + \beta \mathcal{L}_{\text{SSIM}}$$

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

$$\text{MSE} = \frac{1}{MN} \sum_{x=1}^M \sum_{y=1}^N (I_D(x, y) - I_S(x, y))^2$$

```
def combined_loss(pred, target):  
    return 0.8 * mse_loss(pred, target) + 0.2 * ssim_loss(pred, target)  
  
train_loss, train_psnr = train_one_epoch(  
    model, train_loader, optimizer,  
    combined_loss, device)
```



U-Net - Example

Métrica para monitorear la Calidad de las reconstrucciones

$$PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

Loss ↓ ⇒ PSNR ↑

Valores altos de PSNR → imágenes más limpias de ruido

¿Por qué utilizar PSNR si el MSE ya está incluido en la función de pérdida?

El PSNR ayuda a supervisar y comunicar el rendimiento, pero no se utiliza para actualizar los pesos de la red.

Loss → learning

PSNR → evaluation

U-Net - Example

